Ders 3 Genişletilmiş Notu: Kök Bulma Yöntemleri: Grafik, İkiye Bölme, Regula Falsi, Newton-Raphson ve Sekant

Ders 3 Genişletilmiş Notu: Kök Bulma Yöntemleri: Grafik, İkiye Bölme, Regula Falsi, Newton-Raphson ve Sekant

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geçen Haftanın Özeti: Doğrusal Olmayan Denklemler
- 2. Grafik Yöntemi
- 3. İkiye Bölme (Bisection) Yöntemi
- 4. Regula Falsi (Yanlış Nokta) Yöntemi
- 5. Açık Yöntemler ve Newton-Raphson
- 6. Newton-Raphson ile Maksimum Hacim Problemi
- 7. Newton-Raphson Formülünü Taylor Serisinden Çıkarmak
- 8. Sekant (Kiriş) Yöntemi

1. Geçen Haftanın Özeti: Doğrusal Olmayan Denklemler

Hoca derse, geçen hafta başlanan doğrusal olmayan (lineer olmayan) denklemlerin çözümünü kısaca özetleyerek başladı. Sonra sırasıyla üç kapalı yöntemi (grafik, ikiye bölme, regula falsi) ve iki açık yöntemi (Newton-Raphson, sekant) anlattı.

1.1 Bir Denklem Ne Zaman Doğrusal Değildir?

Doğrusal bir denklem verildiğinde çözmek kolaydır. Doğrusal olmayan bir denklemi analitik olarak çözmek ise zordur. Bu yüzden böyle denklemleri sayısal yöntemlerle çözmek çok daha kolaydır. Hocanın slayttaki örneği $x^2 + \sin x + e^{3x}$ ifadesiydi.

Bir denklemin doğrusal olmaması için iki işaret vardır: bilinmeyenin derecesi 1'den büyüktür ya da denklemin içinde trigonometrik veya üstel terimler bulunur. Doğrusal olmayan denklemler bir değişkenli olabileceği gibi, slaytın altındaki örnekte olduğu gibi x ve y içeren iki değişkenli de olabilir. Ancak bu derste görülen yöntemler yalnız tek bilinmeyenli denklemler içindir.

> **Hocanın vurgusu:** İki tane bilinmeyeniniz varsa bu yöntemlerle hiçbir şekilde bir yere varamazsınız. Denklem bir bilinmeyenli olmalıdır.

1.2 Kökün Varlığı, Kapalı ve Açık Yöntemler

Verilen bir $f(x)$ fonksiyonunu bu yöntemlerle çözebilmek için fonksiyonun sürekli olması gerekir. Kapalı yöntemlerde iki sınır verilir ve fonksiyonun $[a, b]$ aralığında sürekli olması istenir. Böyle bir aralıkta en az bir kök olabilir. Aralık çok geniş tutulmuşsa birden fazla kök de olabilir. Ancak seçilen yöntem o anda yalnız bir kökü bulur. Diğer kökü bulmak için a ve b aralığını değiştirmek gerekir.

Hocanın “her şeyden önemlisi” dediği şart şudur: aralığın uçları fonksiyonda yerine konduğunda $f(a)$ ile $f(b)$ ters işaretli olmalıdır.

$$f(a) \cdot f(b) < 0$$

Hoca iki yöntem ailesini şöyle karşılaştırdı:

Özellik	Kapalı yöntemler	Açık yöntemler
Başlangıç	İki sınır: a ve b	Genelde bir başlangıç değeri, bazı yöntemlerde iki
Kökün konumu	Kök iki sınırın arasındadır	İki başlangıç değeri varsa kök bunların arasında olmak zorunda değildir
İterasyon	Daha fazla iterasyon gerekir	Yakınsıyorsa çok daha hızlı sonuca gider
Yakınsama	Genellikle yakınsar	Iraksama ihtimali daha fazladır

1.3 Ortalama Değer ve Ara Değer Teoremleri

Hoca, geçen hafta görülen ortalama değer teoremini hatırlattı. Fonksiyonun a ile b arasında öyle bir c noktası alınır ki, teoremden hesaplanan değer fonksiyonun o noktadaki türevine eşit olur.

Bazı öğrenciler buna neden ihtiyaç duyulduğunu sormuştu. Hocanın cevabı şuydu: Size c noktasındaki türev soruluyor olabilir, ama fonksiyon türevini alamayacağınız kadar karmaşık olabilir. O zaman ortalama değer teoremiyle hesap yaparak, hiç türev işlemiyle uğraşmadan o noktadaki türevi bulabilirsiniz.

Hocanın vurguladığı bir başka nokta, bir fonksiyonun $[a, b]$ aralığında kaç tane ortalama değer noktası olabileceğiydi. Aralıkta kaç salınım noktası varsa, slayttaki teğetlere bakarak o kadar c değeri bulunabilir.

Ardından ara değer teoremi geçildi. Burada $f(a)$ ile $f(b)$ birbirine eşit olmamalıdır. Grafikte $f(a) < f(b)$ de olabilir, $f(a) > f(b)$ de. Aradaki bir c noktasının değerinin hangisinden büyük ya da küçük olduğu grafiğe bakarak söylenebilir.

> **Ek bilgi (derste anlatılmadı):** Bu derste formülleri söylenmeyen iki teorem, ders kaynaklarında büyük olasılıkla şu biçimde geçer. Ortalama değer teoremi: f, $[a, b]$ aralığında sürekli ve (a, b) aralığında türevlenebilirse, $f'(c) = \frac{f(b)-f(a)}{b-a}$ eşitliğini sağlayan en az bir c vardır. Ara değer teoremi: f sürekliyse, $f(a)$ ile $f(b)$ arasındaki her değeri aralıkta en az bir noktada alır. Kök şartının dayanağı da budur, çünkü ters işaretli iki değerin arasında 0 vardır. Ortalama değer teoremindeki c keyfi bir nokta değildir. Bu nedenle formül, türevi yalnız teoremin garanti ettiği c noktasında verir.

1.4 Süreklilik ve Parçalı Fonksiyonlar

Fonksiyonlar sürekli olabildiği gibi parçalı da olabilir. Slayttaki parçalı fonksiyonda $x = 3$ noktasında parçalardan biri kullanılıyordu. Bu fonksiyon parçalı olmasına karşın süreklidir. Başka bir örnekte ise süreksizlik vardı. Hoca böyle bir fonksiyonun köklerinin bulunamayacağını söyledi.

Süreklilik şöyle özetlendi: Bir a noktasında soldan limit, sağdan limit ve fonksiyonun kendi değeri aynıysa, fonksiyonun limiti alınabilir ve fonksiyon o noktada süreklidir. Slayttaki bir grafikte fonksiyon sürekli değildi, ama sağdan ve soldan limitleri alınabiliyordu. Bir diğerinde hem limit yoktu hem de fonksiyon sürekli değildi.

> **Ders sonrası not:** “Süreksiz fonksiyonun kökü bulunamaz” cümlesi bu dersin yöntemleri için geçerlidir. Süreksiz bir fonksiyonun da kökü olabilir, ama işaret değişimine dayanan bu yöntemlerin dayanağı büyük olasılıkla ortadan kalkar.

1.5 İterasyon ve Durma Noktası

Denklemleri iterasyonla çözeceğiz. Hoca bunu bir `for` döngüsüne benzetti: `for` döngüsünde bir son sınır, `while` döngüsünde bir şart nasıl varsa, burada da bir durma noktası gerekir. Slaytta üç ölçüt vardı:

- **Gerçek kök biliniyorsa:** Gerçek değer ile o anda hesaplanan değer arasındaki fark, mutlak değerce belirlenen hatadan küçük olduğu anda durulur.
- **Aralık ölçütü:** n. iterasyonda alt ve üst sınır farkı 2^n'e bölünür ve hatayla karşılaştırılır: $\frac{b-a}{2^n} \leq \varepsilon$.
- **Ardışık iki kök ölçütü:** Bulunan ardışık iki kök değeri arasındaki farkın mutlak değeri hatadan küçük ya da ona eşit olmalıdır: $|x_k - x_{k-1}| \leq \varepsilon$.

Gerçek kök genellikle verilmez. Hoca, örneklerde ve sınavlarda genelde yalnız üçüncü ölçütü beklediğini söyledi. Kod yazarken diğer ölçütler de kullanılabilir, ama elle işlem yaparken hatayı bu şekilde kontrol etmek yeterlidir.

2. Grafik Yöntemi

Hoca kapalı yöntemlere yeniden bir uyarıyla başladı: Sonuç almak daha uzun sürer, ama yakınsadıkları vakit kökü bulurlar. "Sabret, eninde sonunda kökü bulacaksındır." İki başlangıç değeri verilir, fonksiyon bu ikisi arasında sürekli olmalı ve aralıkta bir kök bulunmalıdır.

Çok fazla kapalı yöntem vardır, derste üçü seçildi. Hocaya göre bazı yöntemler birbirine çok benzer. Bir yöntemin uzun süren bir bölümünü kısaltmak için bir şeyler eklenerek yeni bir yöntem ortaya atılmıştır. İlk yöntem, "biraz uzun süren ama ilginç" dediği grafik yöntemidir. Bu bölümün sayısal örneği hocanın önceden çektiği bir video üzerinden, araya girilerek anlatıldı.

2.1 Algoritma

Yöntemin başında iki parametre verilmelidir: başlangıç değeri x_0 ve artım miktarı Δx. Adımlar şöyledir:

1. x_0 fonksiyonda yerine konur, $f(x_0)$ hesaplanır.
2. x değerine Δx eklenir, yeni x fonksiyonda yerine konur.
3. Hesaplanan son iki fonksiyon değeri ters işaretli olana kadar 2. adım tekrarlanır.
4. Ters işaretli durum elde edilince bir iterasyon tamamlanmış olur. Kök, bu sonucu veren iki x değerinin arasındadır.
5. Δx küçültülmelidir, bunun için ikiye bölünür: $\Delta x \leftarrow \Delta x/2$.
6. Bu aralığın sol ucundan başlanarak aynı tarama yapılır.
7. Ters işaretli iki fonksiyon değeri bulunduğunda, bunları veren ardışık iki x arasındaki farkın mutlak değeri hatadan küçük ya da ona eşitse durulur.

Hoca, anlatırken karışık gelebileceğini ama çözüldüğünde bu kadar uzun bir şey yapılmadığının görüleceğini söyledi.

2.2 Örnek: $f(x) = x^2 - 6x + 5$

Verilen: $f(x) = x^2 - 6x + 5$, $x_0 = 1.5$, $\Delta x = 0.75$. Hoca hatayı örneğin sonunda, durma noktasını gösterirken $\varepsilon = 0.0235$ olarak koydu.

1. iterasyon ($\Delta x = 0.75$): İşaret değişene kadar x artırılır.

x	$f(x)$	İşaret değişti mi?
1.5	-1.75	Başlangıç
2.25	-3.4375	Hayır
3	-4	Hayır
3.75	-3.4375	Hayır
4.5	-1.75	Hayır
5.25	1.0625	Evet

Kök 4.5 ile 5.25 arasındadır.

> **Ders sonrası not:** Altyazıya göre hoca $f(1.5) = 1.75$, $f(2.25) = 3.43$ dedi. Doğrusu $f(1.5) = 1.5^2 - 6 \cdot 1.5 + 5 = -1.75$ ve $f(2.25) = -3.4375$'tir. Bu değerler Python ile hesaplandı. İşaret, 4.5 ile 5.25 arasında değişir.

2. iterasyon: $\Delta x = 0.75/2 = 0.375$. Tarama $x = 4.5$'ten başlar, çünkü kök 4.5 ile 5.25 arasındadır. $f(4.5) = -1.75$ zaten bilinmektedir.

- $x = 4.5 + 0.375 = 4.875$ için $f(4.875) = -0.4844$. İşaret değişmedi.
- $x = 4.875 + 0.375 = 5.25$ için $f(5.25) = 1.0625$. İşaret değişti.

Kök 4.875 ile 5.25 arasındadır.

3. iterasyon: $\Delta x = 0.375/2 = 0.1875$. Tarama $x = 4.875$'ten başlar, $f(4.875) = -0.4844$.

- $x = 4.875 + 0.1875 = 5.0625$ için $f(5.0625) = 0.2539$. İşaret değişti.

Kök 4.875 ile 5.0625 arasındadır.

4. iterasyon: Hoca $\Delta x = 0.1875/2 = 0.09375$ değerini 0.094'e yuvarladı. Bu nedenle bundan sonraki x değerleri yuvarlanmış olarak söylendi. Tablolarda hocanın sayıları ve yuvarlamasız değerler birlikte verilmiştir.

İterasyon	Δx (tam)	Δx (hoca)	x (tam)	x (hoca)	$f(x)$ (tam)	$f(x)$ (hoca)
4	0.09375	0.094	4.96875	4.969	−0.1240	−0.1230
4	0.09375	0.094	5.0625	5.063	0.2539	0.2559
5	0.046875	0.047	5.015625	5.016	0.0627	0.06401
6	0.0234375	0.0235	4.9921875	4.9925	−0.0312	−0.0299
6	0.0234375	0.0235	5.015625	5.016	0.0627	0.06401

Her iterasyondaki karar şöyledir:

- **4. iterasyon:** Tarama $x = 4.875$'ten başlar. $f(4.969) < 0$ olduğundan işaret değişmedi. $f(5.063) > 0$ olduğundan işaret değişti. Kök 4.969 ile 5.063 arasındadır.
- **5. iterasyon:** Tarama $x = 4.969$'dan başlar. $f(5.016) > 0$ olduğundan işaret değişti. Kök 4.969 ile 5.016 arasındadır.
- **6. iterasyon:** Tarama yine $x = 4.969$'dan başlar. $f(4.9925) < 0$ olduğundan işaret değişmedi. $f(5.016) > 0$ olduğundan işaret değişti. Kök 4.9925 ile 5.016 arasındadır.

> **Ders sonrası not:** Altyazıda 5. iterasyonun sonucu 0.16 ve 6. iterasyonun alt ucu 4925 olarak geçiyor, kök de bir yerde 5.06 diye söyleniyor. Bağlama ve Python hesabına göre doğrusu sırasıyla 5.016, 4.9925 ve 5.016'dır.

Durma: $|5.016 - 4.9925| = 0.0235 \leq \varepsilon$ olduğundan durulur. Hoca kökü, en son bulunan x değeri olarak söyledi:

$$x_{\text{kök}} \approx 5.016$$

2.3 Sonucu Yorumlamak

Hoca, videoda fonksiyon değerini hatayla karşılaştırabileceğini söylediği yerde araya girdi ve bunun hatalı bir söyleyiş olduğunu belirtti. Gerçek kök bulunsaydı fonksiyon değeri sıfır olurdu. Sayısal yöntemlerde ise her zaman gerçek analitik değer bulunamayabilir, yaklaşık değerler bulunur. Bu yüzden 5.016 fonksiyonda yerine konduğunda sıfır değil 0.064 çıkar. Fonksiyon değeri sıfıra ne kadar yakınsa, karşılığındaki x de köke o kadar yakındır.

Durma kontrolü ise ardışık iki x arasındaki farkın mutlak değeriyle yapılır. Mutlak değer gereklidir, çünkü üstteki değer daha büyük, alttaki daha küçük çıkabilir, fark negatif olabilir ve bu sizi yanıltmamalıdır.

Bu fonksiyonun gerçek kökleri 1 ve 5'tir. 5 yerine konduğunda sonuç 0 çıkar. Hata küçültülürse sonuç 5'e daha da yaklaşır. Hoca hassasiyet için "2 significant digit" örneğini verdi: Virgülün sağındaki ve solundaki basamakların toplamı 2 olacaksa sonuç 5.0 olarak söylenebilir.

> **Ders sonrası not:** Hoca, hata küçültülürse "7. ya da 8. iterasyonda" değerin 5.00032 gibi bir şey olacağını söyledi. Python hesabına göre 8. iterasyonda üst uç 5.0039'dur, 11. iterasyonda 5.00024'tür. Değerin yaklaştığı fikri doğrudur, ama bu sayı 8. iterasyona ait değildir.

Kök olarak genelde en son bulunan x değeri söylenir. Ama işaretin değiştiği iki x arasındaki herhangi bir sayı, örneğin ortadaki bir sayı da kök kabul edilebilir.

2.4 Artım Miktarı ile Hata Aynı Şey mi?

Soru: Fark dediğimiz şey artış miktarı, yani Δx olmuyor mu? 4.99'u Δx kadar artırıyoruz, 5.01'den çıkarınca da Δx'i bulmayacak mıyız?

Cevap: Hayır, bunlar aynı kavram değildir. Δx yeni değer için artım miktarıdır. Hoca bunu şöyle gösterdi: 1 ile 2 arasında alacağın sayılar için $\Delta x = 0.2$ dersem, 1'e sırayla 0.2 ekleyerek ilerlersin. `for` döngüsünde verilen artım miktarı gibidir. ε ise hata yapma payıdır, sonucu ne kadar hatayla kabul edeceğini belirler.

Aralarındaki ilişki yalnız durma anında ortaya çıkar. Tabii ki bir x'ten Δx çıkarırsan bir öncekini, Δx eklersen bir sonrakini bulursun. Ancak üstteki iterasyonlardaki Δx değeri ε'a eşit değildir. Yalnız durulan iterasyonda, bu örnekte 6. iterasyonda, ardışık fark ε'a eşit olur. Bu yüzden biri son kontrolde "Δx, ε'a eşit mi?" diye bakabilir, bir başkası iki kök değerinin farkına bakabilir.

> **Hocanın vurgusu:** Bu kontrolü yalnız fonksiyonların işaretlerinin değiştiği yerde yapabilirsin. 6. iterasyonda 4.969 ile 4.9925'in farkı da Δx'tir. Ama işaret değişmediği için "durdum, köküm 4.9925" dememelisin.

2.5 Kaç İterasyon Yapılacağı Önceden Bilinir mi?

Soru: Buradan yola çıkarak kaç iterasyon yapmamız gerektiğini anlayabilir miyiz? Örneğin 0.0235 olduğunu bilerek 2^{-x}'ten küçük olana kadar diyebilir miyiz?

Cevap: Hoca, işleme başlarken "bu 10 ya da 15 iterasyonda biter" diye bir garanti olmadığını söyledi. Altıncı iterasyonda durulması, hatanın öyle seçilmesindendir. Hata 0.001 yapılırsa altı iterasyonda bitmez. Öyle bir iterasyona gelinir ki ardışık fark ε'a eşit ya da ondan küçük olur, durma noktası orasıdır. Kontrol farklı biçimlerde yapılabilir, örneğin slayttaki $\frac{b-a}{2^n}$ ölçütüyle.

Kodda durma noktasını iterasyon sayısı olarak da belirleyebilirsiniz, örneğin 20 iterasyon. O zaman o iterasyonda durursunuz, ama sonucun hatasını bilmezsiniz. Örneğin 3 iterasyonda durdurmak gerçekçi değerler vermez.

> **Ders sonrası not:** Hoca $\varepsilon = 0.001$ için "7. ya da 8. iterasyonda biter" dedi. Python ile hesaplandığında bu örnek $\varepsilon = 0.01$ için 8. iterasyonda, $\varepsilon = 0.001$ için 11. iterasyonda biter. Öğrencinin sorusunda da doğru bir taraf vardır: Δx her iterasyonda yarıya indiği için durma anındaki iterasyon sayısı $\frac{0.75}{2^{k-1}} \leq \varepsilon$ eşitsizliğinden önceden bulunabilir. Önceden bilinemeyen, her iterasyonda kaç tarama adımı yapılacağıdır.

2.6 Başlangıç Değeri ve Hassasiyet

Hoca köklerin 1 ve 5 olduğunu bildiğini, başlangıç değerini de iterasyon kısa sürsün diye 1.5 seçtiğini söyledi. İlk haftadaki sözünü hatırlattı: Elle çözerken bitsin diye az iterasyonda sonuçlanacak sayısal değerler verilir. Programı yazdığınızda ise, örneğin projede, Δx ve ε değerlerini verebildiğiniz kadar küçük verip çok daha hassas sonuçlar elde edersiniz.

> **Ders sonrası not:** Hoca, başlangıç 3 olsaydı 6 yerine belki 16 iterasyon süreceğini söyledi. Python hesabına göre $x_0 = 3$ ile tarama yine $[4.5, 5.25]$ aralığına gelir, yine 6 iterasyonda biter ve yeni nokta sayısı 13'ten 11'e iner. Bu örnekte 3 daha kötü bir başlangıç değildir. Ayrıca $x_0 = 1.5$, 1 kökünün sağındadır. Tarama sağa doğru gittiği için yöntem 1'i hiç görmez ve 5'i bulur.

3. İkiye Bölme (Bisection) Yöntemi

Bu yöntem bazı kitaplarda aralık yarılama, İngilizce yayınlarda bisection olarak geçer. Adı üstünde, ikiye bölerek ilerlenir. Doğrusal olmayan, analitik çözümü olmayan, bir bilinmeyenli denklemlerin kökünü bulmada kullanılır. Kapalı aralık yöntemi olduğu için iki sınır vardır ve bu aralıkta bir çözüm olduğu gösterilmelidir.

3.1 İlk Kontroller ve Sıfır Çarpım Durumu

Algoritma önce fonksiyona bakmayı ister: Doğrusal mı, onu tanımsız yapan bir x değeri var mı, $[0, 1]$ gibi verilen aralıkta sürekli mi? Hocanın gösterdiği uygulama şudur: Aralığın iki ucu fonksiyonda yerine konur ve işaretlere bakılır. İşaretler tersse aralıkta kök vardır.

$$f(a) \cdot f(b) < 0$$

Slayttaki gösterim "küçük eşit" biçimindeydi. Hoca eşitlik durumunu ayrıca açıkladı: Çarpım sıfır çıkıyorsa ya $f(a)$ ya $f(b)$ ya da ikisi birden sıfırdır. Bu da a ya da b noktasının, veya ikisinin de kök olduğunu gösterir.

> **Ders sonrası not:** Hoca çarpım sıfır çıkarsa "bu aralıkta kök yoktur, uçlar köktür" anlamında konuştu. Doğrusu, çarpımın sıfır olması uçlardan en az birinin kök olduğunu söyler, aralığın içinde başka kök olup olmadığı hakkında bilgi vermez. Ayrıca işaret kontrolü sürekliliği kanıtlamaz. Süreklilik, örneğin fonksiyonu tanımsız yapan bir nokta olup olmadığına bakılarak ayrıca düşünülmelidir.

3.2 Grafik Üzerinde Yöntem

Slaytta siyahla çizilmiş bir $f(x)$ fonksiyonu ve başlangıçta verilen a_1, b_1 değerleri vardı. $f(a_1)$ ile $f(b_1)$ ters işaretli olduğu için aralıkta kök vardır. Yeni nokta, aralığın ortasıdır:

$$c = \frac{a + b}{2}$$

Hoca bu noktaya önce c dedi, sonra hangi uca geldiğine göre adını değiştirdi:

1. $c = \frac{a_1+b_1}{2}$ bulunur. $f(a_1)$ ile $f(c)$ ters işaretli, $f(c)$ ile $f(b_1)$ aynı işaretlidir. Kök a_1 ile c arasındadır. Bu yüzden c artık b_2 olur.
2. $c = \frac{a_1+b_2}{2}$ bulunur. $f(a_1)$ ile $f(c)$ aynı işaretlidir, bu aralıkta kök olamaz. Artık a_1 kullanılmaz ve c artık a_2 olur.
3. $c = \frac{a_2+b_2}{2}$ bulunur. $f(c)$, $f(a_2)$ ile aynı işaretlidir. c artık a_3 olur, kök a_3 ile b_2 arasındadır.
4. $c = \frac{a_3+b_2}{2}$ bulunur. $f(c)$ sıfır ya da yaklaşık sıfır çıkabilir. Hata kontrol edilir.

Hoca aynı şeyi daha sade biçimde de anlattı: Burası a, burası b. İkisinin ortasını al, c çıktı. Kök a ile c arasındaysa yeni b, c olur. Tekrar ortala. Kök o yarıda değilse a'yı bırak, yeni a, c olur. Aralık sıfıra yaklaşana kadar böl. Sıfıra yakın bir değer bulunca "benim köküm budur" dersin.

3.3 Örnek: $f(x) = x^3 - 7x^2 + 14x - 6$, $[0, 1]$

Verilen: $f(x) = x^3 - 7x^2 + 14x - 6$, $a = 0$, $b = 1$, $\varepsilon = 0.01$. Bu bölüm de hocanın videosu üzerinden anlatıldı.

Aralık kontrolü: $f(0) = -6$ negatif, $f(1) = 1-7+14-6 = 2$ pozitiftir. Ters işaretli oldukları için aralıkta kök vardır.

n	a	b	$c = \frac{a+b}{2}$	$f(c)$	Yeni aralık
1	0	1	0.5	-0.625	$[0.5, 1]$
2	0.5	1	0.75	0.9844	$[0.5, 0.75]$
3	0.5	0.75	0.625	0.2598	$[0.5, 0.625]$

n	a	b	$c = \frac{a+b}{2}$	$f(c)$	Yeni aralık
4	0.5	0.625	0.5625	-0.1619	$[0.5625, 0.625]$

İlk iki iterasyon altyazıda okunmadığından ara adımlar eklenmiştir (ara adım eklendi). Karar kuralı her satırda aynıdır: $f(c)$, $f(a)$ ile ters işaretliyse yeni $b = c$, $f(b)$ ile ters işaretliyse yeni $a = c$ olur.

3.4 Durma Ölçütü

Hoca bu soruda farklı bir durma biçimi gösterdi ve bunu her soruda ayrı bir şey göstermek için yaptığını söyledi. Slayttaki kontrol, o andaki aralığın 2^n'e bölünmesiydi:

n	Hesap	Sonuç	≤ 0.01 mi?
1	$\frac{1-0}{2^1}$	0.5	Hayır
2	$\frac{1-0.5}{2^2}$	0.125	Hayır
3	$\frac{0.75-0.5}{2^3}$	0.03125	Hayır
4	$\frac{0.625-0.5}{2^4}$	0.0078	Evet

Bu kontrole göre 4. iterasyonda durulur ve hocanın slaytındaki sonuç şudur: $x_{\text{kök}} \approx 0.5625$. Hoca, hatayı program yazsaydı çok daha küçük verebileceğini ekledi. Sonra bu ölçütü kullanmanıza gerek olmadığını söyledi: Ardışık iki kök değerinin, burada 0.625 ile 0.5625'in, farkına bakmak da yeterli olurdu.

> **Ders sonrası not:** Slayttaki hesapta aralık zaten küçülmüşken yeniden 2^n'e bölünüyor, bu yüzden aralık iki kez küçültülüyor. Ölçüt başlangıç aralığıyla kurulmalıdır: $\frac{b_0-a_0}{2^n} \leq \varepsilon$. Hoca 2. adımı da "0.5 gibi" dedi, slayttaki hesabın sonucu 0.125'tir. Python hesabına göre $[0, 1]$ ve $\varepsilon = 0.01$ için $n = 7$ gerekir ($\frac{1}{128} = 0.0078$). Hocanın önerdiği ardışık fark ölçütü de 7. iterasyonda durur. $|0.625 - 0.5625| = 0.0625$ olduğundan 4. iterasyonda durulmaz. Gerçek kök $2 - \sqrt{2} \approx 0.585786$'dır, 0.5625 bundan 0.0233 uzaktadır, yani istenen hatayı sağlamaz. Devamı şöyledir:

n	c	$f(c)$	$\frac{1}{2^n}$	$\lvert c_n - c_{n-1}\rvert$
5	0.59375	0.0540	0.03125	0.03125
6	0.578125	-0.0526	0.015625	0.015625
7	0.5859375	0.0010	0.0078125	0.0078125

7. iterasyonda durulur, kök yaklaşık 0.586'dır.

> **Ek bilgi (derste anlatılmadı):** Başlangıç aralığıyla kurulan ölçütten gereken iterasyon sayısı önceden hesaplanabilir: $\frac{b_0-a_0}{2^n} \leq \varepsilon$ eşitsizliğinden $n \geq \log_2 \frac{b_0-a_0}{\varepsilon}$ çıkar. Burada $\log_2 100 \approx 6.64$ olduğundan $n = 7$'dir. Kaynaklarda bu sınır büyük olasılıkla ikiye bölmenin en temel özelliği olarak verilir. Yöntem fonksiyonun biçimine bakmaz, aralığı her adımda tam yarıya indirir.

3.5 Yöntemin Özü

Bu yöntemin en önemli özelliği, her yeni $f(c)$ değeriyle a ya da b'nin değişmesidir. Değiştirirken önemli olan, farklı işaret veren f değerleri hangi x noktalarına aitse hep o aralığı küçülterek devam etmektir.

> **Hocanın vurgusu:** Grafik yönteminde de ikiye bölmede de yapılan şey aynıdır: Fonksiyonun ters işaretli olduğu yerlerde aralığı küçültmek ve gittikçe köke yaklaşmak. Belirlenen hataya kadar yaklaşınca durulur ve x bulunmuş olur.

4. Regula Falsi (Yanlış Nokta) Yöntemi

Hoca, bu yöntemin önceki ikisine çok benzediğini, yalnız c noktasının bulunuşunun değiştiğini söyledi. İkiye bölmede c, iki sınır toplanıp ikiye bölünerek bulunur. Regula falsi'de c belirlenirken fonksiyonun $f(a)$ ve $f(b)$ değerleri de kullanılır.

4.1 Yöntemin Fikri ve Adı

Yine doğrusal olmayan, analitik çözümü olmayan, bir bilinmeyenli denklemlerin kökü bulunur. Kapalı aralık yöntemidir, fonksiyon a ile b arasında süreklidir. Eğrinin bir doğruyla yer değiştirmesi sonucunda kökün konumu yanlış belirlenir. Latince "yanlış nokta" anlamına gelen regula falsi adı buradan gelir. İngilizcesi false position olarak geçer.

Fonksiyondaki iki başlangıç noktası bir doğruyla birleştirilir. Bu doğrunun verdiği değer hatalı bir kök değeridir. Kök oymuş gibi düşünülür, sonra değişimlerle köke yaklaşılır.

Adımlar şöyledir:

1. $(a, f(a))$ ve $(b, f(b))$ noktaları arasına bir doğru çizilir.
2. Doğrunun x eksenini kestiği noktaya c denir. c'nin altındaki ve üstündeki iki üçgenin benzerliğinden c hesaplanır.
3. $f(c)$ hesaplanır.
4. $f(c)$'nin işareti $f(a)$ ve $f(b)$ ile karşılaştırılır. Çarpımı sıfırdan küçük olan aralıkla devam edilir.
5. $f(c)$ sıfıra çok yakınsa ya da bulunan iki kök değeri arasındaki fark hatadan küçük ya da ona eşitse durulur.

Slaytta köke yakınsamanın yavaş olmasına karşın yakınsamanın mutlaka olduğu, bu yüzden yöntemin ikiye bölmeye göre daha hızlı çalıştığı yazıyordu. Hoca bunu, daha uzun sürse bile daha kesin sonuç vermesi nedeniyle birçok uygulamada ve projede regula falsi'nin ikiye bölmeye tercih edilmesi olarak yorumladı.

4.2 Grafik Üzerinde Yöntem

Slaytta $f(a)$ ile $f(b)$ ters işaretliydi. Noktalar bir kirişle ya da doğruyla birleştirildi ve doğrunun x eksenini kestiği nokta c_1 oldu. Yanlış nokta denmesinin sebebi, kök aslında başka yerdeyken önce c_1'in kök sanılmasıdır.

c_1 fonksiyonda yerine konur. $f(a)$ ile $f(c_1)$ aynı yönde olduğu için bu aralıkta kök yoktur, $f(c_1)$ ile $f(b)$ farklı yönde olduğu için kök oradadır. Bu kez $f(c_1)$ ile $f(b)$ birleştirilir. Yine gerçek olmayan bir c_2 çıkar. Aynı işlem c_3 için tekrarlanır ve gittikçe köke yaklaşılır.

4.3 Formülün Benzer Üçgenlerden Çıkarılışı

Karşımızda iki üçgen vardır: a, $f(a)$, c_1 köşeli üçgen ve b, $f(b)$, c_1 köşeli üçgen. İkisinin c_1 noktasındaki açıları aynıdır, dolayısıyla tanjantları da eşittir. Tanjant, karşının komşuya oranıdır. $f(a)$ negatif olduğu için karşı kenar $-f(a)$ alınır:

$$\frac{-f(a)}{c_1 - a} = \frac{f(b)}{b - c_1}$$

İçler dışlar çarpımı yapılır:

$$-f(a)\,(b - c_1) = f(b)\,(c_1 - a)$$

Parantezler dağıtılır:

$$-f(a)\,b + f(a)\,c_1 = f(b)\,c_1 - a\,f(b)$$

Bilinmeyen c_1 olduğundan c_1'li terimler bir tarafa alınır ve ortak paranteze alınır:

$$c_1\,\big(f(a)-f(b)\big) = b\,f(a) - a\,f(b)$$

Sonuç, her iterasyonda c adıyla kullanılan formüldür:

$$c = \frac{b\,f(a) - a\,f(b)}{f(a)-f(b)} = \frac{a\,f(b) - b\,f(a)}{f(b)-f(a)}$$

Bulunan c fonksiyonda yerine konur. $f(a)\cdot f(c) < 0$ ise kök bu aralıktadır ve yeni $b = c$ olur. $f(a)\cdot f(c) > 0$ ise bu aralıkta kök yoktur ve yeni $a = c$ olur. Çarpım sıfırsa uçlardan biri köktür. İkiye bölmedeki kontrol birebir aynıdır, tek fark c'nin nasıl hesaplandığıdır.

> **Hocanın vurgusu:** Formülü unutabilirsiniz. Ezberlemeye gerek yok. Fonksiyonu çizin, $f(a)$ ve $f(b)$ noktalarını bir doğruyla birleştirin, tanjant alfaları eşitleyin ve c'yi çekin.

4.4 Örnek: $f(x) = x^3 - 2x^2 - 5$, $[2, 3]$

Verilen: $f(x) = x^3 - 2x^2 - 5$, $[2, 3]$ aralığında en az bir kök olduğu bilinmektedir, $\varepsilon = 0.01$.

> **Hocanın vurgusu:** "Biliyoruz" demesinin bir anlamı yok. Önce a ve b'yi yerine koyup ters işaretli olup olmadıklarına bakın. Ben öyle a, b değerleri verebilirim ki ikisi de fonksiyonu pozitif yapar. Bu kontrolü yapmadan devam ederseniz hiçbir sonuç alamazsınız.

Aralık kontrolü: $f(2) = 8 - 8 - 5 = -5$, $f(3) = 27 - 18 - 5 = 4$. Ters işaretlidirler.

1. adım: $a = 2$, $b = 3$.

$$c = \frac{2\cdot 4 - 3\cdot(-5)}{4-(-5)} = \frac{23}{9} = 2.556$$

$f(2.556) = -1.37$. $f(a)$ da negatif olduğundan a ile c arasında kök yoktur. Kök c ile b arasındadır, yeni $a = 2.556$, $b = 3$ kalır.

2. adım: Aynı formülle $c = 2.669$, $f(2.669) = -0.234$. Yine $f(c)$ ile $f(a)$ aynı işaretlidir, yeni $a = 2.669$, $b = 3$ kalır.

3. adım: $c = 2.6873$, $f(2.6873) = -0.0366$. Yeni $a = 2.6873$, $b = 3$ kalır.

Durma kontrolü: Slaytta burada da $\frac{b-a}{2^n}$ ölçütü kullanıldı:

İterasyon n	Hesap	Sonuç	≤ 0.01 mi?
2	$\frac{3-2.556}{2^2}$	0.1111	Hayır
3	$\frac{3-2.669}{2^3}$	0.0414	Hayır
4	$\frac{3-2.6873}{2^4}$	0.0195	Hayır

Son değer de hatadan küçük çıkmadı. Hoca burada iki seçenek olduğunu söyledi: Hata büyütülebilir, örneğin 0.02 yapılırsa burada durulur. Ya da bir iterasyon daha yapılır. İşi uzatmamak için burada bitirdi. Kök olarak son c değerini aldı, iki ondalığa yuvarladı:

$$x_{\text{kök}} \approx 2.69$$

Hoca, 2.69'un fonksiyonda yerine konunca f değerinin 0.036'dan biraz daha küçük olacağını da söyledi. Python hesabına göre $f(2.69) = -0.0071$'dir.

Ders sonrası not: $\frac{b-a}{2^n}$ ölçütü aralığın her adımda yarıya inmesine dayanır, oysa regula falsi'de aralık yarıya inmez. Bu örnekte $b = 3$ hep sabit kaldı. Burada ardışık c değerlerinin farkına bakmak daha doğrudur. Python hesabına göre $c_4 = 2.6901$ ve $|c_4 - c_3| = 0.0028 \leq 0.01$ olduğundan 4. adımda durulur. Gerçek kök 2.690647'dir, 2.69 sonucu doğrudur.

4.5 İkiye Bölmeyle Karşılaştırma

Hoca, yöntemin ikiye bölmeden tek farkının c'nin hesabı olduğunu yineledi. İşlemler daha uzundur ve daha karışıktır, a ile b'yi toplayıp ikiye bölmek daha kolaydır. Fakat c hesabında $f(a)$ ve $f(b)$ bilgisi de kullanıldığı için daha hızlı yakınsar. İkiye bölmede iterasyon sayısı "atıyorum" 15 ise burada belki 9 olur. Hoca buradaki sayıların yalnız temsilî olduğunu belirtti.

Ders sonrası not: Slaytta "yavaş yakınsama ama daha hızlı çalışma", hocanın sözünde "daha uzun sürse bile" ifadesi geçti. Ders sonunda ise hoca daha hızlı yakınsadığını söyledi. Bu dersin iki örneğinde, ardışık fark ölçütü ve $\varepsilon = 0.01$ ile Python hesabına göre regula falsi daha az adım ister: $x^3 - 2x^2 - 5$ için 4'e karşı 7, $x^3 - 7x^2 + 14x - 6$ için 5'e karşı 7. Her fonksiyonda böyle olması büyük olasılıkla garanti değildir, çünkü aralığın bir ucu sabit kalıp yakınsamayı yavaşlatabilir.

Hoca kapalı yöntemleri burada bitirdi. İki değer a, b vardır, aralık gittikçe sıkıştırılarak sonuç bulunur ve bunlar yakınsayan yöntemlerdir.

5. Açık Yöntemler ve Newton-Raphson

5.1 Açık Yöntemlere Giriş

Açık yöntemlerde bir ya da iki başlangıç değeri olabilir. İki başlangıç değeri varsa kök illa ikisinin arasında olmak zorunda değildir. Yakınsıyorlarsa kapalı yöntemlerden çok daha hızlı sonuca ulaşırlar, ama ıraksayabilirler. Hoca kapalı yöntemlerde ıraksamanın söz konusu olmadığını, çünkü baştan aralıktaki koşulların kontrol edildiğini söyledi.

Üç yöntem anılır: Newton-Raphson, onun türev zorluğunu kaldırmak için geliştirilen sekant ve farklı bir açık yöntem olan sabit nokta (fixed point iteration) yöntemi. Üçüncüsü bu derste anlatılmadı, gelecek haftaya bırakıldı.

Hoca Newton-Raphson'u çok sevdiğini ve birçok mühendislik uygulamasında bir kök bulmaya ihtiyaç olduğunda bu yöntemin tercih edildiğini söyledi. Benzetmesi sarkaçtı: Elinizde bir sarkaç olduğunu düşünün, tutup bıraktınız. Salınır, salınır, salınır ve belirli bir noktada durur. Newton-Raphson'da da sonuçlar önce uç noktalarda, bir bakarsın pozitifte, bir bakarsın negatifte olabilir. Salınımlar yavaşladıkça kök değerleri birbirine çok yaklaşır.

Ders sonrası not: Sarkaç benzetmesi her örnekte görülmez. Python hesabına göre bu dersteki Newton-Raphson örneğinde değerler salınmadan, tek yönden yaklaşır: 0, 0.4286, 0.5697, 0.5856, 0.5858.

5.2 Yöntemin Temel Özellikleri

Newton-Raphson doğrusal olmayan, analitik çözümü olmayan, bir bilinmeyenli denklemlerde kullanılır. Bir $[a, b]$ aralığı verilebilir, ama bulma işlemi sırasında sonuçlar bu aralığın dışına çıkabilir. Çıktı diye kök yoktur dememek gerekir. Sonuçlar ıraksayabilir. Iraksadığı görüldüğü an başlangıç değeri değiştirilmelidir. Yakınsadığında kökü çok hızlı bulur.

Yöntem türev mantığına dayanır, formülde türev ifadesi vardır. İşleme bir başlangıç değeriyle başlanır. Başlangıç değeri verilmemiş, yalnız aralık verilmişse aralıktaki en küçük değer başlangıç alınabilir. Büyüğü de alınabilir, ama genelde küçük olanla başlanır.

5.3 Geometrik Çalışma Biçimi

Hoca grafiği adım adım çizdi:

1. Verilen x_0 fonksiyonda yerine konur, $f(x_0)$ noktası bulunur. Bu noktaya P_0 diyelim.
2. P_0 noktasından fonksiyona teğet çizilir. Teğetin x eksenini kestiği nokta x_1'dir.

3. x_1 fonksiyonda yerine konur. Sıfır vermiyorsa bu bir kök değildir.
4. x_1'in fonksiyonu kestiği noktadan yeni bir teğet çizilir, x eksenini kestiği nokta x_2 olur.
5. Aynı işlem x_3, x_4 için tekrarlanır. x_4 sıfır ya da yaklaşık sıfır verdiyse durulur.

5.4 Formülün Teğetten Çıkarılışı

x_0 ve fonksiyon bilinir, x_1 bilinmemektedir. Teğetin denklemi:

$$y - y_0 = m\,(x - x_0)$$

x_0 noktasındaki eğim m, fonksiyonun o noktadaki türevine eşittir: $m = f'(x_0)$. y_0 ise x_0'ın fonksiyondaki değeridir: $y_0 = f(x_0)$. x_1 noktası x ekseni üzerinde olduğu için orada $y = 0$'dır. Bunlar yerine yazılır:

$$0 - f(x_0) = f'(x_0)\,(x_1 - x_0)$$

Burada x_0, $f(x_0)$ ve $f'(x_0)$ bilinir, bilinmeyen x_1'dir. Önce $f'(x_0)$'a bölünür, sonra x_1 yalnız bırakılır:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

Aynı formül tanjant alfadan da çıkar. Eğim, tanjant alfaya eşittir, tanjant alfa da karşının komşuya oranıdır. Eğim aynı zamanda fonksiyonun x_0 noktasındaki türevidir:

$$\tan\alpha = \frac{f(x_0)}{x_0 - x_1} = f'(x_0)$$

> **Ders sonrası not:** Altyazıya göre hoca karşıyı $x_0 - x_1$, komşuyu $f(x_0)$ diye okudu. Doğrusu, eğim olması için karşı $f(x_0)$, komşu $x_0 - x_1$'dir. Formül yalnız bu şekilde $x_1 = x_0 - f(x_0)/f'(x_0)$ sonucunu verir.

Genelleştirilince $k = 0$'dan başlayan yineleme formülü elde edilir:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \quad k = 0, 1, 2, ...$$

$k = 0$ konur x_1 bulunur, 1 konur x_2 bulunur, 2 konur x_3 bulunur.

5.5 Durma Koşulları

Gerçek kök değeri genelde verilmez. O zaman bulunan son iki x değerinin mutlak farkına bakılır:

$$|x_{k+1} - x_k| \leq \varepsilon$$

Gerçek kök verilmişse bulunan değerle gerçek değerin farkına bakılabilir. Hoca bu durumun garipliğini şöyle anlattı: Gerçek kök veriliyorsa niye bu işlemle uğraşıyorum? Kitaplar bunu da yazar, ama gerçekte karşılaşılacak olan birincisidir. Gerçek kökü bilmediğimiz için bu işlemleri yaparız.

5.6 Örnek: $f(x) = x^3 - 7x^2 + 14x - 6$**,** $[0, 1]$

Verilen: $f(x) = x^3 - 7x^2 + 14x - 6$, $[0, 1]$, $\varepsilon = 10^{-6}$. Başlangıç değeri verilmemiş, aralığın en küçük değeri alınır: $x_0 = 0$. Formül için türev gerekir:

$$f'(x) = 3x^2 - 14x + 14$$

1. adım:

$$x_1 = 0 - \frac{f(0)}{f'(0)} = 0 - \frac{-6}{14} = 0.428571$$

Durma kontrolü: $|0.428571 - 0| = 0.428571 > 10^{-6}$, devam edilir.

2. adım: x_1 formülde yerine konur. $f(0.428571) = -1.20700$, $f'(0.428571) = 8.55102$.

$$x_2 = 0.428571 - \frac{-1.20700}{8.55102} = 0.569724$$

Kontrol: $|0.569724 - 0.428571| = 0.141153 > 10^{-6}$, devam edilir. Hoca bu değerleri 0.42 ve 0.56 diye iki ondalıkla söyledi ve aynı işlemin ardışık fark hatadan küçük olana kadar süreceğini belirtti. Slaytta sonrası yoktu.

Ders sonrası not: Derste hesaplanmayan adımlar Python ile bulundu.

k	x_k	$f(x_k)$	$f'(x_k)$	x_{k+1}	$\lvert x_{k+1} - x_k \rvert$
2	0.569724	-0.111039	6.997622	0.585592	0.015868
3	0.585592	-0.001328	6.830466	0.585786	0.000194
4	0.585786	-1.98×10^{-7}	6.828427	0.585786	2.9×10^{-8}

Fark 10^{-6}'dan küçük olduğu için durulur. Kök 0.585786'dır. İkiye bölmede aynı fonksiyon için çok daha büyük bir hata (0.01) 7 iterasyon istiyordu. Newton-Raphson 10^{-6} hatasını 5 adımda sağladı.

> **Ek bilgi (derste anlatılmadı):** Yöntemin hızı kaynaklarda büyük olasılıkla "karesel yakınsama" adıyla geçer: Köke yakınken doğru basamak sayısı her adımda yaklaşık iki katına çıkar. Yukarıdaki farklarda bu görülür: 0.0159, 0.00019, 2.9×10^{-8}. Yöntemin zayıf noktası ise $f'(x_k)$'nın sıfır ya da sıfıra çok yakın olmasıdır. Teğet yatay olursa x eksenini kesmez, formülde sıfıra bölme olur. Türev küçükse yeni değer çok uzağa fırlayabilir. Hocanın "ıraksarsa başlangıç değerini değiştir" uyarısının bir sebebi büyük olasılıkla budur.

5.7 Programda Türev ve Fonksiyon Nasıl Verilir?

Hoca, öğrencilerin sorabileceği iki soruyu kendisi sordu.

Türevi programa nasıl aldıracağız? Üç seçenek söyledi: Türevi girdi olarak dışarıdan verebilirsiniz, kodun içine kendiniz yazabilirsiniz ya da sayısal türevi kullanabilirsiniz. Sayısal türev ileride görülecek. Ayrıca sekant yöntemi de bu türev zorluğundan kurtulmak için geliştirilmiştir.

Polinomu programa nasıl vereceğiz? Klavyeden string olarak yazıp ayrıştırmaya (parse) gerek olmadığını belirtti.

Soru: Matrislerle ifade edebilir miyiz?

Cevap: Matrislere de gerek yok. Program kullanıcıdan önce fonksiyonun derecesini ister, sonra sırasıyla derece ve katsayıları alır. $x^3 - 7x^2 + 14x - 6$ için üçüncü derece söylenir, ardından terimler verilir:

Derece	Katsayı
3	1
2	-7
1	14
0	-6

Bir terim yoksa, örneğin x'in birinci kuvveti yoksa, katsayısı 0 söylenir ya da hiç söylenmez ve otomatik olarak sıfırlanır. Fonksiyonun bütün bilgisi bu şekilde verilmiş olur.

> **Hocanın vurgusu:** Newton-Raphson'un hiçbir zor tarafı yoktur. Sizi zorlayacak tek şey fonksiyonun türevidir. Trigonometrik ve üstel ifadeler içeren bir fonksiyonun türevini rahat alamayabilirsiniz.

6. Newton-Raphson ile Maksimum Hacim Problemi

Hoca, gerçek dünyada bu yöntemlerle nasıl karşılaşılabileceğine örnek olsun diye bir vize sorusu çözdü ve bunu çok iyi dinlemeyi istedi, çünkü o ana kadar görülen birkaç şey birlikte kullanılıyordu.

6.1 Problem

Kenar uzunluğu 6 olan kare bir kartonun dört köşesinden x uzunluğunda kareler kesiliyor. Kesilen parçalar ayrılınca artıya benzeyen bir şekil kalır. Kenarlar kapatılınca hoca bunu "üstü açık bir kibrit kutusu" diye tarif etti. İstenen, kutunun hacmini en büyük yapan x'tir. x Newton-Raphson ile bulunacak, başlangıç değeri $x_0 = 0$, hata $\varepsilon = 0.01$'dir.

İlk tepki şöyle olabilir: Newton-Raphson bir kök bulma yöntemidir, burada ise maksimum hacim soruluyor, bunların ne ilişkisi var? Hoca aralarında çok güzel bir ilişki olduğunu gösterdi.

6.2 Hacim Fonksiyonu

İki taraftan x çıkarıldığı için kutunun taban kenarı $6 - 2x$, yüksekliği x'tir. Hacim, iki taban kenarı ile yüksekliğin çarpımıdır. Taban kenarları aynı olduğundan:

$$V(x) = (6 - 2x)^2 \, x$$

Kare açılır ve x içeri dağıtılır:

$$V(x) = (36 - 24x + 4x^2)\, x = 4x^3 - 24x^2 + 36x$$

Daha küçük sayılarla çalışmak için her iki taraf 4'e bölünür. Elde edilen ifade $x^3 - 6x^2 + 9x$'tir.

6.3 Asıl Köklenecek Fonksiyon

Bir fonksiyonun maksimum ve minimum noktaları, türevi alınıp sıfıra eşitlenerek bulunur. Bulunan x değerleri minimum ve maksimum noktalarını verir. O hâlde sadeleştirilmiş hacim ifadesinin türevi alınır ve sıfıra eşitlenir:

$$f(x) = 3x^2 - 12x + 9 = 0$$

Newton-Raphson'a girecek fonksiyon artık budur. Formül için bunun da türevi gerekir:

$$f'(x) = 6x - 12$$

Hoca aynı soru "ikiye bölme ile çöz" ya da başka bir yöntemle de sorulabilir, önemli olan bu değil, dedi. Önemli olan, önce hacmi kurmak, sonra türevi sıfıra eşitleyip çözülecek denklemi bulmaktır.

6.4 Çözüm

1. adım: $x_0 = 0$ için

$$x_1 = 0 - \frac{f(0)}{f'(0)} = 0 - \frac{9}{-12} = 0.75$$

Altyazıda bu adım $0 - \frac{3}{-4}$ olarak okunuyor. Denklemin 3'e bölünmüş hâli olan $x^2 - 4x + 3$ ve türevi $2x - 4$ ile aynı sonuç çıkar (ara adım eklendi).

2. adım: $x_1 = 0.75$ için $f(0.75) = 1.6875$, $f'(0.75) = -7.5$.

$$x_2 = 0.75 - \frac{1.6875}{-7.5} = 0.975$$

Hoca buradan sonra işlemin aynı biçimde sürdüğünü söyledi: Bulunan son iki x değerinin mutlak farkı ε'dan küçük ya da ona eşit olana kadar devam edilir. Sonuç 1'e gider, köklerden biri 1'dir.

> **Ders sonrası not:** Slaytta yazılmayan adımlar Python ile hesaplandı. $x_3 = 0.999695$ için $|x_3 - x_2| = 0.0247 > 0.01$ olduğundan devam edilir. $x_4 = 0.99999995$ için $|x_4 - x_3| = 0.0003 \leq 0.01$ olduğundan durulur. Sonuç $x \approx 1$'dir.

6.5 İki Kökten Hangisi Maksimum?

Hoca hâlâ hacmi en büyük yapan x'in ne olduğunu söylemediğini belirtti. Fonksiyon ikinci dereceden olduğu için iki kökü vardır. Diğer kökü bulmak için başlangıç değeriyle oynanmalıdır. Çarpanlara ayırma ile iki kök 1 ve 3 çıkar: $x^2 - 4x + 3 = (x - 1)(x - 3)$.

İki kök hacim formülünde yerine konur:

- $x = 1$: $V = (6 - 2)^2 \cdot 1 = 16$
- $x = 3$: $V = (6 - 6)^2 \cdot 3 = 0$

Hacmin en küçük değeri 0, en büyük değeri 16'dır. Hacmi en büyük yapan kesim $x = 1$'dir.

> **Ders sonrası not:** Hoca "$x_0 = 1.5$ verirsem diğer kökü bulurum" dedi. Python ile hesaplandığında $f(1.5) = -2.25$, $f'(1.5) = -3$ ve $x_1 = 0.75$ çıkar, yöntem yine 1'e gider. $f'(x) = 6x - 12$, $x = 2$'de sıfırdır. 3 kökünü bulmak için başlangıç 2'den büyük seçilmelidir. Örneğin $x_0 = 2.5$ için değerler 3.25, 3.025, 3.0003 biçiminde 3'e gider.

6.6 Yapılan Yaygın Hata

> **Hocanın vurgusu:** Bu soruda öğrencilerin büyük çoğunluğu hacim fonksiyonunun kendisini $f(x)$ alıp Newton-Raphson'a soktu. Bu yanlıştır. Hacmin kökleri maksimum ve minimum noktasını vermez. Hacmin türevini sıfıra eşitlersen, o denklemin kökleri maksimum ve minimum noktalarıdır. Newton-Raphson bu denklem üzerinde, bu denklemin türeviyle kullanılır.

Hocaya göre sorunun değerli yanı budur: Fonksiyon hazır verilmemiştir. Önce hacim hesaplanır, sonra maksimum ve minimum bulma düşünülür, sonra denklem bu dersteki herhangi bir yöntemle çözülür. Hiçbir mühendise çalıştığı yerde "şu fonksiyonun kökü nedir" diye sorulmaz. Zaten bunları bulan hazır araçlar vardır. Asıl iş, problemi denkleme çevirmektir.

7. Newton-Raphson Formülünü Taylor Serisinden Çıkarmak

Hoca, formülü tanjant alfadan çıkardıktan sonra aynısının Taylor serisinden de çıkıp çıkamayacağını sordu. İlk hafta bir öğrenci serilerin neden kullanıldığını sormuştu. Hoca birçok yerde karşımıza çıkacağını söylemişti, bu da onlardan biridir.

Önce gösterimi hatırlattı: x_0'a h eklenirse x_1, x_1'e h eklenirse x_2 olur. Burada h, ardışık iki kök değerinin farkıdır. Yani $x_0 + h = x_1$, başka deyişle $h = x_1 - x_0$.

İlk hafta f_1'i bulmak için f_0'dan yararlanıp terim terim eklenmişti. Taylor serisi şöyledir:

$$f(x_1) = f(x_0) + f'(x_0)\,\frac{x_1 - x_0}{1!} + f''(x_0)\,\frac{(x_1 - x_0)^2}{2!} + \cdots$$

Yalnız ilk iki terim alınır ve $f(x_1)$ sıfıra eşitlenir, çünkü aranan kök noktasıdır:

$$f(x_0) + f'(x_0)\,(x_1 - x_0) = 0$$

Bilinmeyen x_1'dir. İfade dağıtılır ve x_1 yalnız bırakılır:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

Bu, grafikten bulunan formülün aynısıdır. Hoca bunu "ne kadar garip" diye karşıladı: Taylor serisinin ilk iki terimiyle, teğetin eğimine bakarak çıkarılan formülün aynısı elde edilir. "Aslında matematik çok güzel" dedi ve bunun uzun ispatının matematikçilerin işi olduğunu söyledi. Taylor serisinde de ardışık iki kök değeri ve bir türev kullanılarak bir sonraki x bulunmaya çalışıldığı için bu formülden rahatlıkla yararlanılır.

8. Sekant (Kiriş) Yöntemi

8.1 Fikir

Sekant, Türkçede kiriş yöntemi diye geçer. Yine açık aralık yöntemidir, doğrusal olmayan ve analitik çözülemeyen bir bilinmeyenli denklemlerde kullanılır. Newton-Raphson'un formülünde türev vardır. Türev almakta zorlanıyorsan, türev zaman kaybettiriyorsa ya da fonksiyonun türevini alamıyorsan, Newton-Raphson'da türev işlemi kaldırılarak bir değişiklik yapılır. Ortaya çıkan yöntem sekanttır. Fikir, $f'(x_k)$ yerine fonksiyon üzerinde birbirine yakın iki noktadan elde edilen eğim ifadesini koymaktır.

Hoca, sabit nokta yöntemini burada anlatmayıp gelecek haftaya bırakacağını söyledi, çünkü o yöntem bunlara pek benzemiyor.

8.2 Formülün Çıkarılışı

Newton-Raphson'daki grafiğin aynısı çizilir. Bir x_k noktası seçilir ve $f(x_k)$ bulunur. Buna yakın bir başka değer alınır, adı x_{k-1} olsun ve $f(x_{k-1})$ bulunur. Hoca, çizimde rahat gösterilsin diye nokta uzak duruyor olsa da yakın bir değer alındığını belirtti. İki nokta bir doğruyla birleştirilir, bir üçgene tamamlanır ve tanjant alfa hesaplanır. Bu doğru, x_k noktasındaki teğetin eğiminin yerine geçer. Tanjant, karşının komşuya oranıdır:

$$f'(x_k) \approx \tan\alpha = \frac{f(x_{k-1}) - f(x_k)}{x_{k-1} - x_k}$$

Böylece türev, oturup türev alarak değil, bilinen değerler cinsinden bulunur. Hoca "sayısal türev nasıl alınır göstereceğim" dediği şeyin kısaca bu olduğunu söyledi. Newton-Raphson formülünü hatırlayalım:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Paydadaki türevin yerine yukarıdaki ifade yazılır. Payda içindeki paydadaki $x_{k-1} - x_k$ ifadesi yukarı çarpan olarak çıkar. Sonuç sekant formülüdür:

$$x_{k+1} = x_k - \frac{f(x_k)\,(x_{k-1} - x_k)}{f(x_{k-1}) - f(x_k)}$$

Ders sonrası not: Hoca, bu şekilde elde edilen sonucun türevi alıp yerine koyduğunda çıkan değerle aynı olduğunu söyledi. Doğrusu, bu değer türevin yaklaşık değeridir. İki nokta birbirine yaklaştıkça gerçek türeve yaklaşır. Yukarıdaki eşitlikte bu yüzden $\approx$ yazılmıştır.

Bundan sonrası Newton-Raphson'la aynıdır. x_k yerine x_0, x_{k-1} yerine verilen ikinci değer konur, yeni x bulunur, önceki x ile farkına bakılır. Fark ε'dan küçük değilse bulunan değer yerine konur ve devam edilir. Tek fark, türevin yerine geçecek değerin dört işlemle hesaplanmasıdır.

8.3 Örnek: $f(x) = \frac{e^x}{x} - \frac{\sin x}{x^2} + 2x - 10$

Hoca bu örneği "çok zor bir örnek almışım, kendime çok kızdım" diye tanıttı. Bu fonksiyonun türevini almak gerekse, doğru alıp almadığından emin olamazsınız. Türevi yanlış alırsanız Newton-Raphson hep yanlış türevle yanlış sonuca gider. İşte bu yüzden kiriş yöntemi kullanılır.

Verilen: $x_0 = 5$, ona yakın ikinci değer $x_{-1} = 10$, $\varepsilon = 0.01$.

1. iterasyon: Önce fonksiyon değerleri bulunur.

$$f(5) = \frac{e^5}{5} - \frac{\sin 5}{25} + 2 \cdot 5 - 10 = 29.6826 + 0.0384 + 0 = 29.7210$$

$$f(10) = 2212.6520$$

$$x_1 = 5 - \frac{29.7210 \cdot (10 - 5)}{2212.6520 - 29.7210} = 5 - \frac{148.6049}{2182.9310} = 4.931924$$

$|4.931924 - 5| = 0.0681 > 0.01$ olduğundan devam edilir.

Sonraki iterasyonlar: Bulunan değer yerine konarak devam edilir.

k	x_k	$f(x_k)$	$\lvert x_k - x_{k-1}\rvert$	≤ 0.01 mi?
1	4.931924	28.01593	0.068076	Hayır
2	3.813366	9.54950	1.118558	Hayır
3	3.234929	4.33197	0.578438	Hayır
4	2.754669	1.16477	0.480260	Hayır
5	2.578048	0.18478	0.176621	Hayır
6	2.544745	0.00909	0.033303	Hayır
7	2.543023	0.00007	0.001722	Evet

Tablo Python ile hesaplandı. Hoca sonucu şöyle söyledi: Ardışık iki kök değeri arasındaki fark verilen hatanın altına indiğinde durulur, en son bulunan değer de kök kabul edilir:

$$x_{\text{kök}} \approx 2.54$$

8.4 Hesap Makinesinde Radyan Uyarısı

Sinüslü bir ifade hesap makinesiyle hesaplanırken dikkat edilmelidir. Bu örnekte $\sin 5$, radyan cinsinden 5 demektir. Hesap makinesi radyandaysa doğrudan hesaplanır. Dereceyle çalışılacaksa çevrilir: π radyan 180 derecedir, öyleyse 5 radyan kaç derecedir?

$$5 \text{ radyan} = \frac{5 \cdot 180}{\pi} = \frac{900}{\pi} \approx 286.48^\circ$$

Python ile her iki yoldan da $\sin 5 = -0.9589$ çıkar.

> **Ders sonrası not:** Hoca bir yerde "dereceden radyana geçmem gerekiyor" dedi. Slayttaki işlem ise 5 radyanı dereceye çevirmektir, hoca da sonra bunu "dereceye çevirmiş oldum" diye söyledi.
>
> **Hocanın vurgusu:** Sınıfta hesap makinesi kimlerde var? Bunu alışkanlık hâline getirin. Trigonometrik hesapta radyan ayarını unutmayın.

8.5 Hangi Yöntem Seçilmeli?

Hoca son olarak seçim sorusuna değindi. Türevi alabiliyorsanız Newton-Raphson kullanabilirsiniz. "Bunlara hiç bulaşmak istemiyorum, kapalı yöntemleri kullanıp kafam rahat etsin" derseniz kapalı yöntem seçebilirsiniz, ama iterasyon sayınız fazla gelir. Projede ve soruda hangi yöntem istendiyse o kullanılmalıdır.

> **Ek bilgi (derste anlatılmadı):** Sekantın yakınsama hızı kaynaklarda büyük olasılıkla yaklaşık 1.618 mertebesi olarak verilir. Bu, Newton-Raphson'un karesel hızından düşük, ikiye bölmenin her adımda aralığı yarıya indiren doğrusal hızından yüksektir. Bu nedenle sekant, türevi hesaplamak zor olduğunda büyük olasılıkla iyi bir orta yol sayılır. Yukarıdaki tabloda son adımlarda hatanın hızla küçülmesi de buna uyar.

Dersin Toparlaması

Bu derste doğrusal olmayan, tek bilinmeyenli denklemlerin kökünü bulan beş yöntem görüldü. Üçü kapalı yöntemdir: Grafik yöntemi Δx adımlarıyla işaret değişimini arar ve Δx'i yarıya indirir. İkiye bölme aralığın ortasını, regula falsi ise iki ucu birleştiren doğrunun x eksenini kestiği noktayı kullanır. Üçünde de önce aralıkta $f(a) \cdot f(b) < 0$ olduğu kontrol edilir, ters işaretli uçlar korunarak aralık küçültülür. İkisi açık yöntemdir: Newton-Raphson teğetin x eksenini kestiği noktayla ilerler, teğetten, tanjant alfadan ve Taylor serisinin ilk iki teriminden aynı formülle elde edilir. Sekant ise türevin yerine iki noktadan geçen kirişin eğimini koyar. Açık yöntemler yakınsıyorsa çok daha hızlıdır, ama ıraksayabilir.

Maksimum hacim sorusu bu yöntemlerin nasıl kullanılacağını gösterdi: Gerçek problemde önce denklemi kurmak gerekir, kök bulma yöntemi sonraki adımdır. Hoca formülleri ezberlemenin gerekmediğini, çizimden rahatça çıkarılabildiğini, yöntemlerin arka planda birbirine çok yakın olduğunu söyledi. Gelecek hafta sabit nokta iterasyonu görülecek, ardından hocanın hatırladığı kadarıyla denklem takımlarının çözümüne geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.